Base de datos key-value replicada

PCP · Segundo cuatrimestre 2026 · Diseño según consigna V5

1. Estructura e interfaz

La implementación corre en una única VM BEAM. Los procesos no fallan y stop/1 se invoca una vez al final con todos los identificadores. La API pública es start_link/2, stop/1, put/3, put/4, delete/2 y get/2. Claves y valores pueden ser cualquier término Erlang.

start_link(Count, Alfa) valida Count entero positivo y Alfa entero ≥3, crea Count réplicas enlazadas al invocante y un coordinador también enlazado. Antes de retornar espera que las réplicas reciban su configuración. Cada una conoce sus pares y el coordinador. stop/1 monitorea y detiene cada réplica; el coordinador las monitorea y termina cuando terminó la última.

Estado de una réplica

• Peers y Alfa: otras réplicas y cota de almacenamiento.

• Map: Key => {Version, {value, Valor, Vence}}; como máximo Alfa entradas.

• Metadatos: Key => {Version, valor | borrado | desalojado}. Conserva información de una clave aunque ya no conserve su valor.

• Pendientes: ConsultaRef => {Cliente, Ref, Key, Faltan}. No acumula valores de las respuestas.

• Coordinador: identificador del proceso que ordena los cambios.

Estado del coordinador

Pids, Alfa, monitores, Indice y Meta. El índice guarda Key => {Version, Vence} para las entradas residentes; Meta registra versión y estado conocido. El coordinador **no conserva valores de usuario en su estado**. Los valores viajan en mensajes y en los mapas de las réplicas. Para aplicar la política de capacidad construye temporalmente una representación con un marcador fijo sin contenido del usuario.

Los metadatos pueden crecer con las claves conocidas; no se recolectan lápidas porque un mensaje antiguo podría resucitar un borrado. No se usan behaviours OTP, ETS, DETS, Mnesia, persistent_term ni diccionario del proceso. El estado persistente vive en los argumentos de los ciclos de recepción.

2. Protocolo de mensajes

Abreviaturas: I = invocante, C = cliente, R = réplica, P = par, O = coordinador; E = entrada versionada. Cada cliente usa una referencia propia; las consultas remotas usan otra referencia.

Mensaje	Emisor → receptor	Contenido y efecto
peers	I → R	Pares, Alfa, coordinador y referencia de inicio.
{Ref, Pid}	R → I	Confirma recepción de configuración antes de retornar start.
put, delete	C → R	Pedido con cliente, referencia y datos.
escritura	R → O	Reenvía pedido y origen; O asigna versión y decide.
ordenada	O → todas R	Entrada y acción única: guardar, borrar, rechazar o ignorar.
{Ref, ok}	R origen → C	Confirma al aplicar localmente, sin esperar a los pares.
ofrecer	R → cada P	Ofrece valor vigente antes de descartarlo.
propuesta_oferta	P → O	Solicita procesar oferta en el orden común.
propuesta_replica	R → O	Ordena una entrada interna recibida para pruebas del protocolo.
get	C → R	Pedido de lectura por clave.
barrera_get / get_ordenado	R ↔ O	Ordena el inicio de lectura respecto de pedidos previos.
consultar	R → P	Pide información local de una clave.
barrera_consulta / consulta_ordenada	P ↔ O	Ordena la respuesta local respecto de eventos anteriores.
respuesta_consulta	P → R	Devuelve información local de la clave.
candidato, fin	R → C	El cliente reduce candidatos y termina con comprobación de TTL.
stop	I → R	Detiene la réplica.
DOWN	VM → I / O	Informa terminación a stop y al coordinador.

Los mensajes inesperados se descartan. Los tipos peers de tres y cuatro campos también se aceptan en pruebas internas con pares controlados; el inicio público usa configuración con confirmación. El coordinador no espera que las réplicas confirmen los eventos distribuidos.

3. Orden, conflictos y convergencia

Todo put o delete público se deriva al coordinador, que asigna una versión con erlang:unique_integer([monotonic]). También recibe las ofertas que los pares proponen. Procesa cada mensaje de a uno, calcula una única transición sobre su índice/metadatos y envía a todas las réplicas el mismo evento con esa decisión.

Una réplica no vuelve a elegir la víctima ni a evaluar qué entradas estaban vencidas al decidir. Aplica la acción ordenada y, si recibió el pedido del cliente, responde después de su aplicación local. La operación no espera que el resto aplique el evento: la replicación sigue siendo asincrónica.

Consistencia observable y orden común

La consistencia eventual se evalúa desde afuera: cuando cesan las escrituras y se propagan los cambios, los nodos deben responder de forma consistente y correcta. La consigna no exige por sí misma que sus estados internos sean idénticos. Este diseño utiliza mapas iguales como estrategia para lograr esa propiedad; otros diseños podrían resolver las lecturas con estados físicos diferentes. El TTL sigue aplicándose en el instante de cada lectura.

Todos los eventos mutables de la API pública y de ofertas salen del mismo coordinador hacia cada réplica. Erlang preserva el orden por pareja emisor/receptor. Las réplicas parten del mismo estado y aplican las mismas acciones con las mismas entradas; por inducción sus mapas coinciden luego de aplicar el mismo prefijo de eventos. Si cesan las escrituras, llegan los mensajes y las ofertas terminan, alcanzan el mismo mapa. Los resultados transitorios pueden diferir mientras unas réplicas avanzan más que otras.

Para put(d) que desaloja a, seguido de delete(d) desde otra réplica, todas reciben y aplican primero la decisión de desalojo y luego la lápida. Una oferta que llegue después puede recuperar una copia si el coordinador la acepta, pero la recuperación también es un evento común. No se exige que distintos intercalados de ejecuciones separadas elijan siempre el mismo resultado final.

Versiones y borrados

Las versiones se comparan con los metadatos conocidos. Una versión mayor gana; una menor se ignora. Las operaciones originadas por el coordinador tienen versiones únicas. Las copias y ofertas conservan la versión de origen.

Un delete crea {Version, borrado} y quita el valor. La lápida impide que un put anterior lo restaure; una nueva escritura sí puede reemplazarla. Desalojar deja {Version, desalojado}: es ausencia local, no delete. Si llega una copia de la misma versión, el almacén puede recuperarla; el selector de respuestas prefiere ese valor a la marca de desalojo de igual versión.

El orden común fundamenta la igualdad de estado bajo los supuestos y el protocolo público descritos; los tests finitos no demuestran todos los intercalados. El selector protege residentes más entrante: no reconstruye valores ya desalojados tras un borrado.

4. TTL, capacidad y ofertas

La V5 (§4.7) establece: «el 75 % escrito más recientemente (en el nodo local) siempre siga disponible». §4.6 establece: «nunca se devuelve un valor vencido». §4.7 también dice: «No es válido tener entradas adicionales en alguna estructura secundaria, α es el tope de valores permitidos para un nodo.»

El TTL se calcula al ordenar el put como vencimiento absoluto con el reloj monotónico de la VM. infinity no vence; 0 nace vencido. TTL inválido falla en el invocante antes del envío. Replicar u ofrecer conserva el instante. El cliente verifica vigencia al terminar get; ningún valor vencido se devuelve.

Decisión de diseño: conservar físicamente las entradas protegidas aunque venzan. Se toma Q=ceil(3·Alfa/4), redondeado hacia arriba. Estos detalles no son citas literales de la consigna. La recencia considerada por el selector es la de residentes más entrante; no hay cupos independientes según el origen de la escritura. Vencer no crea una lápida; un delete explícito sí puede quitar una entrada protegida.

Selección y capacidad

Para una clave existente se aplica la versión aceptable sin ocupar otra plaza. Con espacio se incorpora incluso un valor vencido. Con el mapa lleno, se consideran residentes y entrante: las Q versiones mayores quedan protegidas. Entre las restantes se elige un vencido, prefiriendo la menor versión si hay varios; si no hay vencidos, se elige la menor versión. El entrante puede descartarse.

La elección se calcula una sola vez en el coordinador, con el reloj en ese instante. Su evento identifica la víctima; las réplicas la retiran antes de insertar y nunca almacenan Alfa+1 valores en Map. Una oferta puede entrar con vacante, reemplazo o un vencido no protegido, sin desalojar vigentes.

Ofertas y terminación

Cada réplica ofrece un valor vigente a sus pares antes de descartarlo. Una réplica receptora envía la propuesta al coordinador, que determina su admisión y distribuye el resultado en el flujo común. La aplicación de una oferta no produce nuevas ofertas; una oferta no desaloja un valor vigente. Por cada escritura hay una cantidad finita de réplicas y mensajes de oferta; ninguna oferta dispara otra cadena.

Los índices y metadatos del coordinador no incluyen valores de usuario; los mensajes en tránsito sí los transportan. Las ofertas dan oportunidad de conservación, no la garantizan: si todas las réplicas descartan una entrada elegible, puede perderse. Liberar espacio más tarde no recupera una copia perdida.

5. Lecturas, progreso y validación

Un get pasa primero por una barrera no bloqueante en el coordinador: la réplica reenvía el pedido y sigue en su ciclo. Al recibir get_ordenado, si tiene un valor o lápida, envía un candidato y fin al cliente. Si la clave es desconocida o desalojada y hay pares, registra la consulta y pregunta a todos.

Cada par deriva la consulta por una barrera al coordinador y luego responde con su información local, sin consultar a terceros. Estas barreras ordenan las respuestas respecto de eventos anteriores; el coordinador no espera respuestas de réplicas. La réplica solicitante acepta una respuesta por cada par esperado, reenvía candidatos al cliente y conserva solo referencias y pares faltantes. Al llegar la última, envía también su estado local actualizado y fin.

El cliente elige la mayor versión entre candidatos y comprueba el TTL justo antes de retornar {ok, Valor} o not_found. Una copia de la misma versión prevalece sobre una marca desalojado. Los valores de respuestas no se retienen en Pendientes. Los mensajes de candidatos y fin se envían desde una única réplica en ese orden.

Progreso y restricciones

Ni las réplicas ni el coordinador esperan sin atender mensajes a otra réplica: la espera ocurre en el cliente. No hay una dependencia circular entre los procesos para contestar consultas. Bajo los supuestos de procesos sin fallas y entrega eventual, terminan las consultas; no se promete latencia acotada ni tolerancia a la caída del coordinador.

Pruebas y límites

rebar3 compile y rebar3 eunit usan advertencias como errores. La suite cubre API, TTL, versiones, borrados, capacidad, ofertas y consultas. Por API pública se prueban puts y delete concurrentes sobre una clave, lecturas simultáneas con escritura y convergencia posterior. Pruebas internas complementarias controlan mensajes atrasados, ofertas y réplicas pausadas.

Una cantidad finita de pruebas no puede recorrer todos los intercalados, cargas, tiempos de TTL ni términos Erlang. La justificación para escrituras y ofertas es el orden único del coordinador y la aplicación de sus decisiones en cada réplica; para lecturas, referencias independientes y respuestas locales sin espera circular. El argumento supone procesos sin fallas, entrega eventual y procesamiento de mensajes. No demuestra formalmente cada implementación ni garantiza latencia acotada. La interpretación de la cota para datos en tránsito queda sujeta a la consigna.